CV MIDTERM STUDIO

Computer Vision Study Notes

Lectures 2–5 · Szeliski Chapters 1–3 · Trask Chapters 1–5

34 focused units with worked examples and exam traps.

Start with the original sample to identify weak steps. Review those topics, then cover the wider lectures and assigned reading.

Prepared from your supplied files, 8 October 2026. The sample demonstrates a format; it does not predict the actual exam.

Contents

1. OpenCV pipelines: shapes and information
2. System design and believable evaluation
3. What the learning signal changes
4. Forward prediction, dot products and matrix orientation
5. Linear classifiers: shapes, bias and geometry
6. Summed multiclass hinge loss and margins
7. Softmax, cross-entropy and confidence
8. Regularization as a preference between fitted models
9. Scalar gradient descent: residual, derivative and update
10. Learning several weights and outputs together
11. Activation functions and the path of the gradient
12. Hidden features, forward passes and network capacity
13. Backpropagation through gates and branches
14. Vector backpropagation and numerical checks
15. CNN convolution: operation, shape and shared parameters
16. Pooling, receptive fields and translation behavior
17. CNN training: batch norm, regularization and evaluation
18. CNN architectures: identify the mechanism, not just the name
19. Scope, inverse problems, and evidence
20. Classification challenges and what an image distance measures
21. Nearest neighbors, boundary behavior, and honest model selection
22. Homogeneous primitives and incidence
23. Transformation hierarchy and 3D rotation choices
24. Projection, calibration, and lens distortion
25. Lighting, reflectance, and why intensity is ambiguous
26. Optics: focus, exposure, and peripheral effects
27. Digital sensing, noise, sampling, and aliasing
28. Color, gamma, and compression without mixing conventions
29. Point operations, alpha compositing, and tone distributions
30. Linear, separable, directional, and recursive filtering
31. Nonlinear denoising, morphology, and regions
32. Fourier reasoning for filters and restoration
33. Interpolation, pyramids, wavelets, and multiband blending
34. Warping, antialiasing footprints, and morphing

Eight visual calculations follow the notes.

Sample exam coverage

OpenCV pipelines: shapes and information

Trace the shape, channel order, numeric range and retained evidence after each operation.

Track what each operation changes

Default `cv2.imread` returns a color image in BGR order. A normal image array has shape (height, width, 3). `cv2.resize(img, (width, height))` uses the reverse order for its size argument. A grayscale conversion usually returns an array of shape (height, width), representing one intensity channel.

```
img = cv2.imread("leaf.jpg")
if img is None:
    raise FileNotFoundError("leaf.jpg")
img = cv2.resize(img, (224, 224))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blur, 100, 200)
```

The image after resize has shape (224,224,3). Grayscale and edges usually have shape (224,224). Canny thresholds concern gradient evidence. They are neither class probabilities nor a selected interval of pixel brightness.

Explain the mechanism and the tradeoff

Resize standardizes geometry but can remove small lesions or distort shape when aspect ratio is changed. Grayscale simplifies the input and can reduce dependence on color variation, but loses useful discoloration cues. Gaussian blur reduces rapid noise fluctuations before derivatives, but can erase narrow real boundaries. Edge maps can help with contour information while discarding color and much texture.

Choose transformations because they address a data problem. Compare them using validation data. More preprocessing is not a guarantee of higher classification accuracy.

Worked example

Resize a 640×480 image to a 256×256 input without stretching or cropping.

1. Use scale=min(256/640,256/480)=0.4.
2. The resized image is 256×192.
3. The remaining 64 vertical pixels form32 pixels of padding above and below.
4. Use the same deterministic policy at inference.

Result: Resize to(width,height)=(256,192), then pad top/bottom by 32 pixels.

Exam traps

- RGB2GRAY is inappropriate for an image still in default BGR order.
- An explicitly supplied positive Gaussian kernel size must be odd.
- A clean-looking or edge-only image may discard the signal needed for classification.
- Validate image loading before passing the result to cvtColor.

▶ **Check your understanding**

Sources: Original sample Questions 1 and 4 · OpenCV image smoothing and Canny documentation

---

Sample exam coverage

System design and believable evaluation

Connect dataset organization, preprocessing, model choice and evaluation to the deployment setting.

Define what counts as a new example

For waste classification, labels must distinguish Plastic, Paper, Glass and Other consistently. Check class counts, label errors and near-duplicate captures. Several photographs of the same item or the same capture session may be dependent. Keep these together when splitting, so test performance measures the intended generalization.

Split before making data-dependent choices

A 70/15/15 split of 2,000 images gives 1,400 training,300 validation and300 test examples. Stratify where possible and preserve dependency groups. An 80/20 split can also work if validation is drawn from training or training-only cross-validation handles tuning.

Fit learned normalization statistics using training data. Generate augmentation after splitting and apply it to training examples. Choose transformations, hyperparameters and models using validation. Keep the final test labels out of these choices.

Choose metrics that reveal failure

Accuracy is useful when the class distribution and costs make it meaningful. For imbalance, add per-class precision, recall, macro-F1 and a confusion matrix. Precision concerns the reliability of positive predictions; recall concerns how many real positives you found. Macro-F1 averages each class's F1 equally.

Recognize two different failure patterns

A training/test accuracy gap suggests overfitting, while good in-domain scores and poor new-phone scores suggest domain shift. Inspect both data and evaluation before changing architecture. Background shortcuts, camera response, lighting and capture conditions can all cause failures.

Worked example

A defect detector has$TP=30, FP=10, FN=20$. Calculate precision, recall and F1.

1. Precision$=30/(30+10)=0.75$.
2. Recall$=30/(30+20)=0.60$.
3. $F1=2\times0.75\times0.60/(0.75+0.60)=2/3\approx0.6667$.

Result: Precision75%, recall60%, F1≈66.7%. State defective as the positive class.

Exam traps

- Do not tune repeatedly against the final test set.
- A random split does not fix dependency leakage or shared background shortcuts.
- High accuracy can hide failure on a rare but important class.
- Neither raw scores nor normalized softmax values establish calibration.

▶ **Check your understanding**

Sources: Original sample Questions 2 and 5 · Lecture 2 validation/cross-validation · Lecture 5 training and augmentation

---

Assigned reading

What the learning signal changes

Separate the learning objective from how the model stores what it learns.

Chapters 1–2 supply the framework for the later calculations: understand the operations that make a prediction, compare that prediction with a learning signal, and change the stored model. Merely running prediction repeatedly does not train a network.

Question	Distinction	Consequence
Where does the signal come from?	Supervised learning uses target information. Unsupervised learning seeks structure without those category targets.	An image can be used in either setting; input format does not decide supervision.
How is the model represented?	A fixed architecture has a fixed parameter count. A nonparametric method can grow its representation with the data.	A basic kNN stores examples; an MLP stores a fixed collection of coefficients.

These axes are independent. An unsupervised model can be parametric. Nonparametric does not mean that nothing is stored or that there are literally no numerical quantities.

A feedforward prediction uses current input and learned parameters. Previous inference inputs are not automatically recurrent state. Separately, a low training loss establishes fit to the training examples; held-out evidence is needed for generalization.

Worked example

A fixed 3072→100→10 MLP is trained on twice as many images. Which model quantities necessarily double?

1. The number of examples doubles.
2. The layer dimensions and connections are unchanged.
3. Therefore the parameter count does not double; data storage and work per epoch can increase.

Result: Dataset size doubles; the fixed model parameter count stays unchanged.

Exam traps

- Supervised/unsupervised and parametric/nonparametric are different distinctions.
- A cache used for backward is not a recurrent memory of prior inference inputs.
- Zero training error is not proof of held-out accuracy.

▶ **Check your understanding**

Sources: Trask Ch. 1, printed pp. 3–8 · Trask Ch. 2, printed pp. 10–19 · Lecture 4 slides 17–21

Assigned reading

Forward prediction, dot products and matrix orientation

Track which coefficients belong to each input and output before performing the arithmetic.

With one output, the model adds the contributions of its features. With several outputs, each output owns a separate weight vector. Store one vector per row under the column-vector convention:

```
p = Wx                 W: C×D, x: D×1, p: C×1
p_j = Σ_k W_jk x_k
P = XW^T               X: N×D, P: N×C
```

Do not mix the row-batch convention with the column-vector convention. A transpose changes orientation, not the feature–weight pairing. Gradients later must use the same convention.

The dot product is a useful template response, but its magnitude depends on vector lengths as well as alignment. It is not automatically a normalized similarity or a probability. A large feature can dominate even if its coefficient is smaller than another coefficient. Permuting features preserves the response only when the coefficients are permuted identically.

Chapter 3 also composes predictions. If there is no nonlinear activation between the maps, the composition remains linear: $W_2(W_1x)=(W_2W_1)x$. This prepares the reason Lecture 4 inserts an activation.

Worked example

Compute the book's three-input prediction with x=[8.5,0.65,1.2] and w=[0.1,0.2,0].

1. Pair the feature with its own coefficient.
2. Contributions: 8.5·0.1=0.85; 0.65·0.2=0.13; 1.2·0=0.
3. Sum the three contributions.

Result: 0.98.

Exam traps

- A zero coefficient ignores that feature in the present forward pass but can still receive a gradient.
- A linear output need not lie between 0 and 1.
- A compatible shape is necessary, but feature semantics must match as well.

▶ **Check your understanding**

Sources: Trask Ch. 3, printed pp. 24–46

Lecture coverage

Linear classifiers: shapes, bias and geometry

Read the same classifier as a matrix, a bank of templates and a collection of hyperplanes.

```
s = Wx + b              W: C×D, b: C
predicted class = argmax_j s_j
augmented x = [x; 1]    augmented W = [W b]
```

Algebraically, each row of W computes one class score. Visually, reshape that row into an image-sized template. Geometrically, equality of two class scores defines (w_a−w_b)·x+(b_a−b_b)=0. The normal comes from the difference of class weights; a class's own zero-score plane need not be a decision boundary.

The bias translates the boundary. The bias trick absorbs it into a coefficient of a constant feature. It does not make an affine map nonlinear. Scaling obeys f(cx)=cf(x)+(1−c)b; the lecture's simpler

identity assumes no bias.

A single binary boundary cannot isolate XOR, an annulus or several separated modes. Extra depth without nonlinearities still collapses to an affine classifier.

Clarification: flattening keeps the values and a recoverable ordering. A fully connected layer lacks enforced locality and weight sharing; flattening itself does not delete pixel indices. Its parameter count grows with image size: C(D+1) with one bias per class.

Worked example

For x=[56,231,24,2], compute the dog response to [1.5,1.3,2.1,0] with bias 3.2.

1. The four products are 84,300.3,50.4,0.
2. Add the products to obtain 434.7.
3. Add the class bias 3.2.

Result: Dog score = 437.9.

Exam traps

- Argmax uses scores; softmax preserves their ordering.
- One class template cannot flexibly represent every orientation or mode.
- Without scaling the bias too, scaling W alone need not scale every affine score.

▶ **Check your understanding**

Sources: Lecture 3 slides 8–32 · Lecture 5 slides 3–4,13–14

Lecture coverage

Summed multiclass hinge loss and margins

Correct class is highest is weaker than correct class beats every rival by the margin.

```
L_i = Σ_(j≠y_i) max(0, s_j − s_yi + Δ)
L_data = (1/N) Σ_i L_i
```

Compute one term for each incorrect class. Negative margin expressions contribute zero. A positive expression contributes its entire value. A correctly classified example can still have positive loss if the score gap is too small.

Minimum data loss is zero, while the loss is unbounded above. With all C scores equal and $\Delta=1$, the summed loss is $C-1$. Including the correct class adds the constant Δ, rather than a useful constraint. Averaging class terms instead of summing rescales the data-loss term and changes its balance with an unchanged penalty.

Once all margins are satisfied, small score changes may leave the data loss exactly zero. The zero-loss weights need not be unique. If every bias-free score gap already meets Δ, scaling all weights upward preserves zero loss. With affine scores, that argument scales the full affine parameterization, including biases. Regularization can express a preference between such solutions.

Worked example

Use the lecture's first score column $[3.2, 5.1, -1.7]$; the correct class is the first entry (Cat), and the margin is 1.

1. Second entry (Car): $5.1-3.2+1=2.9$.
2. Third entry (Frog): $-1.7-3.2+1=-3.9$.
3. Apply $\max(0,\cdot)$ to each expression and add.

Result: 2.9. The mean of the lecture's three example losses is $(2.9+0+12.9)/3=5.266666\ldots$.

Exam traps

- Do not include the correct class in the sum.
- Do not confuse a correct argmax with a satisfied margin.
- A sum over classes and a mean over examples are separate operations.

▶ **Check your understanding**

Sources: Lecture 3 slides 40–58

Lecture coverage

Softmax, cross-entropy and confidence

Normalize scores, choose the target's probability, and retain the correct logarithm convention.

```
p_j = exp(s_j) / Σ_k exp(s_k)
L_i = -ln(p_yi) = ln(Σ_k exp(s_k)) - s_yi
stable softmax: replace every s_j by s_j - max_k s_k
```

The denominator makes the class probabilities sum to one. It also means every rival affects the loss even though the one-hot expression selects only the target's log probability. Increasing the target score relative to rivals reduces the loss.

Adding the same constant to all scores changes nothing. Multiplying them by a positive factor preserves argmax but changes confidence. A high probability can still be poorly calibrated; normalization supplies a probability model, not a calibration certificate.

For C equal logits, p_y=1/C and L=ln C. **Lecture 3 slide 84 has a sign typo:** its −log(C) should be log(C); the stated log(10)≈2.3 numerical example is right. For more than one class and finite logits, cross-entropy is strictly positive; its infimum is zero, approached as p_y→1. Unlike hinge loss, it has no finite zero-loss plateau.

For sigmoid with binary cross-entropy, the combined derivative with respect to the output logit is p−y. The sigmoid derivative must not be multiplied in a second time.

Worked example

Logits are [0,ln 2,ln 3] and the third class is correct.

1. Exponentiate to obtain [1,2,3].
2. Normalize by 6: the correct-class probability is 0.5.
3. Use natural logarithms: −ln 0.5.

Result: L=0.693147… .

Exam traps

- For one-hot targets, evaluate −ln p for the correct class, not for the predicted class.
- Use ln C, not −ln C, for uniform initialization.
- Subtracting a common constant and scaling logits have different effects.

▶ **Check your understanding**

Sources: Lecture 3 slides 70–90 · Lecture 4 slides 22,29

Lecture coverage

Regularization as a preference between fitted models

Read λ relative to the data-loss scale and distinguish different regularization mechanisms.

```
L_total = L_data + λR(W)
R_L2(W) = Σ_j W_j²     ∂(λR_L2)/∂W_j = 2λW_j
R_L1(W) = Σ_j |W_j|
```

Data loss asks the model to fit the observed targets. A penalty expresses additional preferences, such as limiting coefficient magnitude or favouring simpler fitted behavior. Its purpose is to reduce unwanted fitting of noise and improve generalization; making training performance worse is not an end in itself.

L2 penalizes concentrated large coefficients more strongly. For inputs [1,1], [1,0] and [0.5,0.5] give the same score, but squared L2 norms are 1 and 0.5. Their L1 norms are both 1. L1 can encourage sparsity; its derivative at zero requires a subgradient convention.

Check whether a question uses $\lambda\|W\|^2$ or $(\lambda/2)\|W\|^2$. The derivative differs by two. Also check whether the data loss is a sum or mean; the same numerical λ has a different relative effect if only the data term is rescaled.

Dropout and augmentation do not simply add the same scalar penalty. They change training behavior or examples. Batch normalization can have a regularizing effect but also directly changes the parameterization and optimization.

Worked example

Use $R=0.1(1^2+(-2)^2)$. Compute the penalty and its gradient vector.

1. Squared norm is $1+4=5$.
2. Penalty is $0.1\cdot5=0.5$.
3. Differentiate without a 1/2 factor: $2\cdot0.1\cdot[1,-2]$.

Result: Penalty 0.5; gradient $[0.2,-0.4]$.

Exam traps

- Changing a sum to a mean can change the effective regularization strength.
- Weight count alone does not determine generalization.
- The convention for penalizing biases must be stated; many problems count them but do not regularize them.

▶ **Check your understanding**

Sources: Lecture 3 slides 59–69 · Lecture 5 slide 23

Assigned reading

Scalar gradient descent: residual, derivative and update

Keep the error value, signed residual and parameter gradient distinct.

```
p = wx
δ = p − y
E = δ²                  dE/dw = 2δx
L = δ²/2                dL/dw = δx
w_new = w − η(dL/dw)
```

The book displays $E=\delta^2$ but uses weight_delta=δx. This corresponds to the derivative of half-squared loss; equivalently, the missing factor two can be absorbed into the learning-rate convention. In an exam, use the explicit loss and update given, rather than silently switching conventions.

Multiplication by x performs the book's three effects. A zero input stops that weight's update because it cannot affect the prediction. A negative input reverses the required direction. A larger input changes sensitivity and can cause much larger updates.

The derivative gives local sensitivity. Subtracting it is a local descent direction, but a large finite step can overshoot. A learning rate less than one is not a universal safety rule. For a repeated scalar example with half-squared loss, weight error evolves by a factor $1-\eta x^2$; convergence requires $0<\eta<2/x^2$ when $x\neq 0$.

Hot-and-cold testing moves by a fixed increment after comparing nearby errors. Its step cannot automatically shrink with error and can oscillate near the optimum.

Worked example

Use x=0.5,w=0.5,y=0.8 and the book update with η=0.1.

1. p=0.25; δ=−0.55; displayed squared error E=0.3025.
2. weight_delta=δx=−0.275.
3. w_new=0.5−0.1(−0.275).

Result: w_new=0.5275. For full squared-error derivative with η=0.1 instead, w_new=0.555.

Exam traps

- A positive squared error says nothing about the required sign of the weight change.
- The full square has derivative 2δx; the half-square has derivative δx.
- Training is not guaranteed to reach zero error for arbitrary data, architecture or step size.

▶ **Check your understanding**

Sources: Trask Ch. 4, printed pp. 48–76

Assigned reading

Learning several weights and outputs together

Each weight gets its destination's residual times its source's activation.

```
p = Wx                  δ = p − y
L = (1/2)Σ_j δ_j²
∂L/∂W_jk = δ_j x_k      ∂L/∂W = δxᵀ
W_new = W − ηδxᵀ
```

Multiple inputs share one residual if they feed the same output. Multiple outputs sharing one input use different residuals but the same input value. The outer product combines the two cases. Its orientation must match the weight matrix: rows are outputs and columns are inputs here.

A zero input blocks its coefficient's data gradient; a zero coefficient does not. Freezing a coefficient deliberately sets its update to zero, even if the true gradient is nonzero. The other coefficients may compensate, but the model's adjustable directions are reduced.

Printed-code corrections: pp. 91–93 contain inconsistent snippets. For multiple outputs use input times delta, not input times weights. Preserve delta[i] rather than overwriting the residual vector with a scalar. With output rows, use outer(delta,input), rather than an input-row outer product. The equations and destination/source indexing resolve these issues without copying flawed snippets.

Chapter 5's digit templates help interpret learned positive and negative contributions. A large dot product is a class response; plain weighted sums are not constrained probabilities.

Worked example

Take $x=[8.5,0.65,1.2]$, $w=[0.1,0.2,-0.1]$, $y=1$ and $\eta=0.01$ for one output.

1. $p=0.85+0.13-0.12=0.86$; $\delta=-0.14$.
2. Gradient vector is $[-1.19,-0.091,-0.168]$.
3. Subtract η times each corresponding gradient.

Result: New weights: $[0.1119,0.20091,-0.09832]$.

Exam traps

- An outer product's orientation depends on how W is stored.
- The output's target must appear through its residual.
- Do not recompute later gradients using weights already updated in the same nominal simultaneous step.

▶ **Check your understanding**

Sources: Trask Ch. 5, printed pp. 80–98 · Lecture 4 slide 30

Lecture coverage

Activation functions and the path of the gradient

Nonlinearity changes representational power; local slopes change trainability.

Function	Value	Local derivative	Main concern
Sigmoid	1/(1+e^(−z))	p(1−p)	Small slope at extreme z; output not zero-centred
tanh	tanh(z)	$1-a^2$	Zero-centred but still saturates
ReLU	max(0,z)	1 for z>0, 0 for z<0	Negative branch can block learning
Leaky ReLU	max(0.01z,z)	1 or 0.01 away from zero	Allows a small negative-side slope

A ReLU's nonlinearity comes from selecting different branches. A stack of affine layers without an intervening nonlinear function remains affine, even if there are biases. Sigmoid or tanh saturation gives very small local derivatives, which multiply along a chain. ReLU avoids that saturation on its positive branch, but its negative branch is flat.

A dead unit stays inactive across the relevant data, rather than merely producing zero on one example. Leaky ReLU addresses the flat negative side. The lecture also names GELU, $g(z)=z\Phi(z)$, for smooth activation gating used in newer architectures.

At z=0, ReLU is nondifferentiable: the implementation selects a convention. The formulas above are for points strictly on one branch.

Worked example

A sigmoid output is p=0.8 and upstream gradient is −2. Find the input gradient.

1. Local derivative is 0.8·0.2=0.16.
2. Apply the chain rule: local derivative times upstream.
3. Keep the negative sign of the upstream gradient.

Result: −0.32. Five consecutive local derivatives of 0.1 would instead attenuate a unit upstream signal to 10^−5.

Exam traps

- Do not call ReLU non-saturating everywhere.
- The sign of a single weight does not determine whether a neuron is active.
- A probability-producing output activation and a hidden activation serve different purposes.

▶ **Check your understanding**

Sources: Lecture 4 slides 9,12–14,28 · Lecture 5 slide 20

Lecture coverage

Hidden features, forward passes and network capacity

A hidden layer learns several responses that the output layer can combine.

```
h = ReLU(W₁x+b₁)
s = W₂h+b₂
parameter count for D→H→C = DH + H + HC + C
```

The hidden layer is a learned feature map. Different responses can support different appearances of one class, while the output combines them. The template interpretation is useful, but hidden units need not be literal named parts.

A constructive XOR example on binary inputs uses $h_1=\mathrm{ReLU}(x_1+x_2)$, $h_2=\mathrm{ReLU}(x_1+x_2-1)$, and output h_1-2h_2. Its four outputs are 0,1,1,0. Removing the nonlinear operations would collapse this into an affine map and lose that behavior.

For the lecture's 3072→100→10 network, affine-layer parameter counts are 307,300 and 1,010, totaling 308,310. The input and ReLU layers add no learned coefficients.

Architecture follows the structure and objective: MLP for fixed vectors, CNN for grids, recurrent state for sequences, Transformer attention for tokens or patches, autoencoder for compression/reconstruction, GAN for adversarial generation, and Siamese shared encoders for pairwise comparison.

For the survey terms in slide 20, distinguish their mechanisms: an RBF network uses radial-basis responses for function approximation; a self-organizing map forms a neighborhood-preserving unsupervised map; a deep belief network stacks stochastic layers; capsule networks represent part–whole relationships; and spiking networks use event-like activations for neuromorphic settings. These are architecture concepts, not interchangeable synonyms for a standard MLP.

Worked example

Run the spam example x=[1,0,1] with H_1 weights [0.5,−0.2,0.3], H_2 [0.4,0.1,−0.5], output weights [0.7,0.2], and all biases zero.

1. Hidden preactivations are 0.8 and −0.1.
2. ReLU gives [0.8,0].
3. Output logit is 0.7·0.8+0.2·0=0.56.
4. Sigmoid gives the model's spam probability.

Result: p=0.6364525…; threshold 0.5 predicts spam.

Exam traps

- Count both biases, not just matrix weights.
- A larger network can overfit; smaller size is not the only possible regularization choice.
- The input layer is usually not counted as a learned affine layer.

▶ **Check your understanding**

Sources: Lecture 4 slides 4–22

Lecture coverage

Backpropagation through gates and branches

Cache forward values, multiply along each path and add at shared inputs.

```
upstream = ∂L/∂output
input gradient = local derivative × upstream
```

Node	Backward behavior
a+b	Send the upstream gradient unchanged to both inputs.
ab	Send g·b to a and g·a to b.
max(a,b)	Route the gradient to the unique winning branch; specify a tie convention.
One value reused	Add every branch's contribution at that value.

Backpropagation applies the chain rule in reverse graph order. The local derivative must use the cached forward values. A variable reused in several places has several routes to the loss, so overwriting rather than summing its gradient loses information.

Backpropagation computes parameter gradients. The optimizer then uses those gradients to change the parameters. These are distinct operations.

In the spam example, sigmoid plus BCE supplies $p-y=-0.363547\ldots$ at the output logit. The first output connection gets this times 0.8. The inactive hidden unit blocks its upstream signal at ReLU. A zero input feature blocks only its own connection gradient.

Worked example

Backpropagate $f=(x+y)z$ with $x=-2, y=5, z=-4$.

1. Forward: $q=x+y=3$ and $f=qz=-12$.
2. Seed $\partial f/\partial f=1$.
3. Multiply backward: $\partial f/\partial q=z=-4$ and $\partial f/\partial z=q=3$.
4. Add backward: $\partial f/\partial x=\partial f/\partial y=-4$.

Result: Forward $f=-12$; gradients for (x,y,z) are $(-4,-4,3)$.

Exam traps

- The upstream gradient may be negative; max routing does not take its absolute value.
- The multiply gate uses the other input.
- Shared inputs require summed gradients.
- A cached value from a different forward pass gives an incorrect derivative.

▶ **Check your understanding**

Sources: Lecture 4 slides 23–29,31,33

Lecture coverage

Vector backpropagation and numerical checks

A gradient has its variable's shape, and a numerical reference can expose a wrong backward pass.

```
y = Wx+b                 g = ∂L/∂y
∂L/∂W = gx^T             ∂L/∂x = W^T g
∂L/∂b = g                for one example
central difference ≈ [L(w+h)−L(w−h)]/(2h)
```

For batches, contributions to a shared parameter add. If the loss is a mean, its averaging factor must be included exactly once. Shape compatibility helps diagnose transposes, but it is not a proof that the derivative represents the right computation.

Numerical checks perturb one coefficient and rerun the forward function. They are too expensive for routine training but useful on a small smooth test case. Compare the numerical and analytic gradients with an absolute or relative error measure that handles near-zero values sensibly.

The lecture suggests h≈10^−5 and relative error below 10^−7. These are useful heuristics, not universal correctness guarantees: precision, magnitude, finite-difference cancellation and nondifferentiable hinges/ReLUs can change the expected discrepancy. Do not check across an active-branch change without interpreting it.

In a normal per-batch training loop, clear accumulated parameter gradients before the next backward pass. Intentional gradient accumulation must scale or interpret the effective update consistently.

Worked example

Use $x=[1,-2]^T$, $W=[[3,4],[-1,2]]$, and $g=[0.5,-1]^T$.

1. Form gx^T row by row.
2. The first row is $[0.5,-1]$; the second is $[-1,2]$.
3. Compute W^Tg for the input gradient.
4. Sum each input's contributions from the two outputs.

Result: $\partial L/\partial W=[[0.5,-1],[-1,2]]$; $\partial L/\partial x=[2.5,0]^T$; $\partial L/\partial b=[0.5,-1]^T$.

Exam traps

- Do not include the mean-loss factor twice.
- An analytic derivative may disagree with a central difference at a kink.
- Backward() accumulates gradients unless the code clears them.

▶ **Check your understanding**

Sources: Lecture 4 slides 30–33

Lecture coverage

CNN convolution: operation, shape and shared parameters

A filter is local spatially, spans the input depth and is reused across output positions.

```
O = floor((W−K+2P)/S)+1       apply separately to H and W
ordinary conv parameters = C_out(K²C_in+1)
output depth = number of filters = C_out
```

A standard CNN layer computes cross-correlation: multiply a patch with the displayed filter and sum, adding its bias. Mathematical convolution flips the kernel along both spatial axes. A fixed-kernel calculation must state which operation is intended.

For a color image, one ordinary filter has depth C_in; it sums spatial and channel contributions to one output feature map. Different filters give different channels. The kernel coefficients and one bias are reused at every position, so resolution changes computation without changing this parameter count.

Stride changes the spacing of sampled windows. Padding changes what windows fit at the border. For odd K and stride 1, symmetric $P=(K-1)/2$ preserves width. The simple formula assumes no dilation.

A shared coefficient participates in many output positions, so its backward gradient sums all of those contributions: each contribution is the upstream gradient at that output times the corresponding patch value. The filter bias similarly sums upstream values. An input pixel receives contributions from every window that used it. Input-gradient expressions can involve flipped kernels, but stride and padding must also be accounted for; copying the forward operation blindly is insufficient.

Slide 11 clarification: its table rejects $W=7,K=3,S=3,P=0$ as a non-integer exact fit, but the stated floor formula gives $O=2$. Standard floor sizing uses only fully fitting windows and leaves a border remainder. If an exam instead requires exact tiling, state that convention explicitly.

Worked example

A 32×32×3 input meets ten 5×5 filters, stride 1, padding 2, with one bias per filter.

1. Spatial output: floor((32−5+4)/1)+1=32.
2. Depth is ten, one channel per filter.
3. Each filter has 5·5·3+1=76 parameters.
4. Reuse these parameters at every output position.

Result: Output 32×32×10; 760 trainable parameters.

Exam traps

- Do not multiply parameter count by output area.
- Kernel depth equals input channels for an ordinary dense-channel convolution.
- Padding is per side, so the spatial formula uses 2P.
- Output depth is controlled by filter count, not input RGB depth.

▶ **Check your understanding**

Sources: Lecture 5 slides 4–16,24 · Lecture 4 slides 28,30

Lecture coverage

Pooling, receptive fields and translation behavior

Track spatial coverage and downsampling separately from the number of channels.

Max and average pooling reduce spatial resolution independently in each channel and ordinarily add no trainable coefficients. A 2×2 stride-2 pool maps 16×16×32 to 8×8×32. Max-pool backward routes gradients to the selected maxima; ties need a convention.

```
r₀=1, j₀=1
r_l = r_(l−1) + (K_l−1)j_(l−1)
j_l = j_(l−1) S_l
stride-1, equal kernels: r = 1 + L(K−1)
```

Here r is receptive-field width in original-input coordinates and j is spacing between neighboring feature positions in those coordinates. Downsampling increases j, so later kernels expand the receptive field faster. This is theoretical coverage, not a claim that every covered pixel has equal learned influence.

Global average pooling averages each channel's entire spatial plane. It maps 7×7×512 to 512 values and can greatly shrink the final fully connected head.

Translation equivariance means a translated input produces a corresponding translated feature map, subject to boundary and stride effects. Classification invariance means the output does not change. Convolution alone does not guarantee that final invariance; local pooling offers limited tolerance, not perfect invariance to every shift.

Worked example

Track the receptive field through conv 3×3 S1 → pool 2×2 S2 → conv 3×3 S1.

1. Start r=1,j=1.
2. First conv gives r=3,j=1.
3. Pool gives r=4,j=2.
4. Final conv gives r=4+2·2=8,j=2.

Result: The output unit can depend on an 8×8 input region.

Exam traps

- Three 3×3 stride-1 kernels have receptive field 7, not 9.
- Pool preserves channels unless another operation changes them.
- Flatten counts spatial values times channels, excluding batch size.
- Equivariance and invariance are different claims.

▶ **Check your understanding**

Sources: Lecture 5 slides 17–21,29,34

Lecture coverage

CNN training: batch norm, regularization and evaluation

Keep learned coefficients, batch statistics and evaluation choices distinct.

```
x_hat = (x−μ_B)/sqrt(σ_B²+ε)
y = γx_hat+β
W_new = W−η∇_W L
```

Convolutional batch normalization forms mean and variance for each channel over the batch and spatial positions. Its learned scale and shift contribute two affine parameters per channel. During evaluation, the lecture's model uses running statistics collected during training, rather than current evaluation-batch statistics. The usual course order is Conv→BN→ReLU.

Weight decay or an L2 penalty limits coefficient magnitude. Dropout masks activations during training; under inverted dropout, kept values are divided by their keep probability. Augmentation produces plausible label-preserving training variants. Whether a flip, crop or color change preserves the target is task-dependent.

A training step runs forward scores, loss, backward gradients and an optimizer update. Momentum combines past and present gradient information; a learning-rate schedule changes the step-size multiplier. The lecture's OneCycle schedule warms up and then decays.

For model selection, monitor training and validation performance. Repeatedly adjusting a model using the test set converts that test set into validation evidence. Keep an untouched test assessment for the final report. For a small transfer-learning dataset, the lecture suggests freezing the pretrained backbone and training a new classifier; more data can justify fine-tuning more layers with a small learning rate.

Worked example

One BN channel has $x=10, \text{mean}=6, \text{variance}=4, \varepsilon=0, \gamma=2, \beta=-1$.

1. Standardize: $(10-6)/\sqrt{4}=2$.
2. Scale and shift: $2\cdot 2-1$.
3. Separate these learned affine parameters from the mean/variance statistics.

Result: Output is 3. With 64 channels, γ and β add 128 trainable affine parameters.

Exam traps

- Do not add a softmax before the CrossEntropyLoss shown in the lecture.
- Do not accidentally leave BN/dropout in training mode for ordinary evaluation.
- Do not tune repeatedly on the final test split.
- The meaning of p in dropout must be fixed as drop probability or keep probability.

▶ **Check your understanding**

Sources: Lecture 5 slides 20–24,39 · Lecture 4 slides 16,22,33

Lecture coverage

CNN architectures: identify the mechanism, not just the name

Connect each family to the problem its design addresses and interpret the course experiment cautiously.

Family	Course-relevant mechanism
LeNet	Early learned convolution and pooling hierarchy before a classifier.
AlexNet	ReLU, dropout, augmentation and GPU computation helped train a larger network.
VGG	Stack small 3×3 kernels to grow coverage with additional nonlinear transformations.
GoogLeNet / Inception	Parallel branches at different scales; 1×1 bottlenecks reduce costly channel depth; concatenate outputs.
ResNet	Learn $F(x)=H(x)-x$ with $y=x+F(x)$, making identity-like solutions and gradient paths easier.
DenseNet	Concatenate earlier features for reuse; this is different from residual addition.
MobileNet	Depthwise spatial filtering followed by 1×1 channel mixing.
EfficientNet	Coordinate depth, width and input-resolution scaling.
ViT	Embed image patches and use attention over tokens with position information.
ConvNeXt	Modernized convolutional design with depthwise kernels, LayerNorm and GELU.

A 1×1 convolution is a learned channel map at each spatial position, not a spatial neighborhood filter. An Inception bottleneck can reduce the channels before a 3×3 or 5×5 operation. Residual addition

requires compatible tensors; a projected shortcut can match spatial resolution and channel depth.

Degradation refers to worse training fit as a plain network gets deeper. It differs from overfitting, where training fit may be good but held-out behavior deteriorates. Residuals help optimization; they do not guarantee the best result in every shallow, short-training experiment.

The slide 38 CIFAR comparison reports a specific subset/CPU/short-training setup. SimpleCNN has fewer parameters and higher accuracy than its LeNet-style baseline. VGGMini's 70.1% exceeds ResNetMini's 63.7% in that setup; this does not establish a universal ranking of the architecture families.

Worked example

Reduce channels before a 5×5 convolution: 28×28×192 → 1×1 with 16 outputs → 5×5 with 32 outputs. Ignore biases and preserve spatial size.

1. Direct 5×5→32 cost is $28^2 \cdot 192 \cdot 25 \cdot 32 = 120{,}422{,}400$ multiplies.
2. The 1×1 reduction costs $28^2 \cdot 192 \cdot 16 = 2{,}408{,}448$.
3. The reduced 5×5 costs $28^2 \cdot 16 \cdot 25 \cdot 32 = 10{,}035{,}200$.
4. Total bottleneck cost is 12,443,648; divide direct by bottleneck.

Result: About 9.677 times fewer multiplies for this specified design; both outputs have shape 28×28×32.

Exam traps

- Concatenation adds channels; addition matches existing channels entry by entry.
- Equal receptive field does not mean equal function.
- A shortcut cannot add incompatible shapes without a matching operation.
- Course experiment accuracy is evidence for that setup, not a universal architecture constant.

▶ **Check your understanding**

Sources: Lecture 5 slides 25–39

Assigned reading

Scope, inverse problems, and evidence

Know what the assigned sources cover, why pixels underdetermine a scene, and how to evaluate a proposed method.

Coverage distinction: Lecture 2 teaches image classification, datasets, nearest-neighbor methods, and validation. Szeliski Chapters 2 and 3 add image formation and processing; those are assigned-reading coverage here. No claim is made that a topic is likely to appear on the exam merely because it is in the book.

Assigned source	Substantive coverage in these units
Szeliski Ch. 1, §§1.1-1.5	Inverse vision, applications and constraints, scientific/statistical/engineering/data approaches, history context, dependency overview, column-vector notation. The syllabus and additional-reading lists are context rather than extra examinable algorithms.
Szeliski Ch. 2, §2.1	Points, lines, planes, conics/quadrics as representations; 2D/3D transforms and rotations; orthographic/perspective projection; intrinsics/extrinsics; depth and planar mappings; lens distortion.
Szeliski Ch. 2, §§2.2-2.3	Lighting, BRDF/diffuse/specular/Phong/global illumination, optics, sensing/noise, sampling/aliasing, color, gamma, compression and PSNR.
Szeliski Ch. 3, §§3.1-3.3	Point/color operators, alpha compositing/matting, histograms/tonal adjustment, linear/separable/oriented/recursive filters, nonlinear/bilateral/guided filtering, morphology, distances, connected components.
Szeliski Ch. 3, §§3.4-3.6	Fourier/DFT/FFT/DCT and restoration, interpolation/decimation, Gaussian/Laplacian pyramids, wavelets/lifting/blending, parametric/mesh warping and morphing.
Lecture 2	Semantic gap and nuisance variation, datasets and top-5, L1/L2 and kNN boundaries/costs, hyperparameters, train/validation/test and cross-validation, dimensionality and learned features.

Think of vision as estimating latent causes from observations. Geometry maps a 3D scene to viewing rays; illumination and reflectance determine outgoing light; optics and the sensor map this into pixels. Several causes can produce the same observation. A dark patch may be dark material, a shaded surface, or low exposure. A single perspective image also cannot distinguish proportional changes in object size and distance.

A physics model exposes ambiguity, a prior or learned representation adds information, and evaluation tells you whether the resulting assumptions work. Specify the output and the conditions first. Use synthetic inputs with a known answer, introduce controlled noise, then test representative real images. A plausible picture alone is weak evidence of correctness.

Chapter 1's history describes a movement from restricted scene models toward feature/statistical methods and then large-scale learned recognition. Learn this progression as context; the practical link is that large datasets, computation, and learning methods changed what could be estimated reliably.

```
Column-vector convention: apply A first, then B -> B A x.
Finite homogeneous point: (u,v,w), w != 0 -> (u/w,v/w).
Same observations do not necessarily imply the same scene.
```

Worked example

Why can two differently sized objects have identical perspective images?

1. Model u=fX/Z and v=fY/Z.
2. Scale every object coordinate and depth by the same factor c.
3. The ratios cX/(cZ) and cY/(cZ) do not change.

Result: Absolute size/depth cannot be recovered from that single projection without extra information.

Exam traps

- Do not infer lecturer emphasis from a reading chapter.
- A method can look plausible while violating a known synthetic answer.
- Historical descriptions in this 2021 draft are not statements about the latest state of the field.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 1, printed pp. 1-32 (PDF pp. 27-58) · CV_Lecture_2.pdf, PDF pp. 1-83

Lecture coverage

Classification challenges and what an image distance measures

Relate nuisance variation to the semantic gap, know dataset roles, and distinguish top-1 from top-5.

Classification assigns an image to a category, but the numeric array contains no explicit category semantics. The same object changes pixels with viewpoint, illumination, deformation, occlusion, and

background. Different examples of one class vary internally; fine-grained classes may differ only subtly. A useful classifier must respond to category evidence while tolerating irrelevant variation.

A raw-pixel representation is sensitive to location. Translating an image can move every edge onto different coordinates and make an otherwise identical object far away by L1/L2 distance. Occlusion changes the evidence available; invariance is useful only for transformations that preserve the target label. Learned visual features can support kNN by making their distance reflect task-relevant similarity more closely.

Lecture example	What to retain
MNIST	28×28 grayscale handwritten digits; ten classes. The slides use a 50k training convention and 10k test set, so keep that convention separate from other dataset splits.
CIFAR-10 / CIFAR-100	32×32 RGB; 10 versus 100 classes; both slides describe 50k training and 10k test examples. A flattened image has 3072 values.
ImageNet lecture setting	1000 categories, variable image sizes, a validation split, and top-5 evaluation. These are the lecture's challenge setting, not every version of ImageNet.
Places / Omniglot	Scene categories / character categories used to discuss few-shot learning.

Top-1 asks whether the highest-ranked predicted class is correct. Top-5 asks whether the correct class is among the five highest-ranked choices. Detection additionally localizes objects; captioning predicts a structured text sequence. Classification can act as a building block, but the required outputs differ.

Worked example

The true class is ranked fourth. What do top-1 and top-5 record?

1. Top-1 examines rank 1 only.
2. Top-5 examines ranks 1 through 5.

Result: Top-1 failure, top-5 success.

Worked example

How many coordinates are in one flattened CIFAR RGB image?

1. 32×32 spatial sites.
2. Three channel values per site.

Result: 32×32×3=3072.

Exam traps

- A small pixel distance need not mean semantic similarity.
- The target label decides which transformations are relevant invariances.
- Top-5 success is not the same as identifying the correct highest-ranked label.

▶ **Check your understanding**

Sources: CV_Lecture_2.pdf, PDF pp. 2-31 · CV_Lecture_2.pdf, PDF pp. 76-82

Lecture coverage

Nearest neighbors, boundary behavior, and honest model selection

Compute distances and votes, explain complexity and dimensionality, and select k without contaminating the test set.

1-NN stores training examples and returns the label of the closest example. Unweighted kNN finds the k closest and takes a class vote. L1 creates distance contours aligned with coordinate axes; L2 uses Euclidean geometry. The chosen metric can change the neighbor ordering and prediction. Features with different scales can dominate either metric; any learned preprocessing statistics belong to the training/development procedure.

```
L1(a,b) = sum_j |a_j-b_j|
L2(a,b) = sqrt(sum_j (a_j-b_j)^2)
Brute-force distance search for one query: O(ND)
Stored numeric values: O(ND)
```

The slide's O(1) training shorthand assumes retaining a reference to already-loaded arrays. Copying/loading N vectors of D features costs O(ND); no optimization of classifier weights is needed. Prediction is summarized as O(N) when D is treated as fixed. kNN boundaries may have small irregular regions caused by isolated examples. Larger k often smooths the boundary and reduces outlier sensitivity, but can wash out real small regions. An odd k does not eliminate all multiclass ties.

k and distance metric are hyperparameters. Use training data to fit/store examples and validation data to choose them. Cross-validation rotates a held-out development fold and averages results; keep the final test data outside that choice. Querying training examples against a set that includes themselves is resubstitution: distinct points retrieve their own labels at zero distance, so it cannot establish generalization.

Uniform coverage becomes expensive as dimension grows: with m locations along each of D independent coordinates, a grid requires m^D points. Lecture 2 uses this curse of dimensionality to explain why a formal approximation claim with infinitely many examples does not make raw-pixel kNN practical. Learned features can supply a more useful lower-complexity notion of similarity.

Worked example

For q=(0,0), compare A=(3,0) and B=(2,2).

1. L1(q,A)=3; L1(q,B)=4.
2. L2(q,A)=3; L2(q,B)=√8≈2.828.

Result: L1 picks A; L2 picks B.

Worked example

Nearest labels are A,B,B,A,B for k=5.

1. Count two A votes and three B votes.
2. Choose the class with most votes.

Result: B.

Worked example

How should you compare k=1,3,5 fairly?

1. Fit/store using training data.
2. Evaluate candidate k values on validation or development cross-validation.
3. Freeze the choice; evaluate final performance on untouched test data.

Result: Select with validation; reserve test for the final estimate.

Exam traps

- Squared L2 has the same ranking as L2 but a different numeric value.
- A tie-breaking policy must cover distance ties and class-vote ties.
- Repeated test-driven selection can produce optimistic accuracy.
- Do not interpret the slide's O(1) statement as zero data-loading or storage cost.

▶ **Check your understanding**

Sources: CV_Lecture_2.pdf, PDF pp. 32-70 · CV_Lecture_2.pdf, PDF pp. 71-82

Assigned reading

Homogeneous primitives and incidence

Use projective coordinates to unify finite points, lines, parallel intersections, and 3D planes.

A homogeneous point is an equivalence class of nonzero vectors: multiplying all coordinates by a nonzero scalar changes the representation, not the point. A finite 2D point (u,v,w) becomes (u/w,v/w); w=0 represents a direction at infinity. The all-zero vector is excluded.

```
2D line l=(a,b,c): a x+b y+c=0, or l^T (x,y,1)=0.
Line through p,q: l=p×q.
Intersection of lines l1,l2: p=l1×l2.
3D plane m=(a,b,c,d): aX+bY+cZ+d=0.
3D line through p,q: r=(1-lambda)p+lambda q.
```

Normalizing a line by $\sqrt{a^2+b^2}$ gives a unit normal; its constant is an oriented quantity, and the unsigned origin distance is $|c|/\sqrt{a^2+b^2}$. Parallel lines have an ideal homogeneous intersection. In 3D, a line is conveniently represented by two points or a point and a direction, although an infinite 3D line has four geometric degrees of freedom and the unrestricted two-endpoint form is redundant. Restricting lambda to [0,1] gives the segment.

Conics and 3D quadrics have quadratic homogeneous equations $p^T Q p = 0$ and provide compact curve/surface representations. This chapter introduces them as primitives; it does not require deriving their full multi-view calibration theory. Incidence constraints are the useful common thread: a point is on a line/plane when the appropriate inner product vanishes.

Worked example

Normalize (12, −6, 3).

1. Divide all finite Cartesian coordinates by w=3.

Result: (4, −2).

Worked example

Where do x=1 and x=3 meet projectively?

1. Use l1=(1,0,−1), l2=(1,0,−3).
2. Their cross product is (0,2,0), proportional to (0,1,0).

Result: The vertical ideal point; no finite Cartesian intersection.

Exam traps

- Homogeneous coordinates are not normalized by the sum of their entries.
- A zero last coordinate cannot be converted by dividing.
- Avoid treating an oriented line constant as a universally positive distance.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §2.1, printed pp. 36-39 (PDF pp. 62-65)

Assigned reading

Transformation hierarchy and 3D rotation choices

Track what each transformation preserves, apply composition in the correct order, and understand constrained rotation representations.

2D transform	Form	DoF	Guaranteed geometric preservation
Translation	$x'=x+t$	2	Orientation, lengths, angles, parallel lines, straight lines
Rigid/Euclidean	$x'=Rx+t$	3	Lengths and angles
Similarity	$x'=sRx+t$	4	Angles and common relative scale
Affine	$x'=Ax+t$	6	Parallelism and straight lines
Projective	$p'\propto Hp$	8	Straight lines; parallelism need not remain

For 2D Cartesian x-right/y-up coordinates, R(theta)=[[cosθ,−sinθ],[sinθ,cosθ]]. In an image whose y axis points down, the visible rotation direction differs; always state axes. A 3×3 homogeneous representation makes translation part of matrix multiplication. Applying A then B means BA for column vectors. For nonsingular transforms, invert that matrix to undo the mapping.

In 3D the corresponding degrees of freedom are 3,6,7,12,15. A proper rotation obeys $R^TR=I$ and $\det(R)=+1$. Nine matrix entries therefore do not mean nine independent orientation parameters. 2D lines and 3D planes are covectors: for $p'=Hp$, their equation coefficients transform by H^{-T} so that incidence is preserved. A 3D line has a different representation.

Euler angles are order-dependent and can have singularities. Axis-angle uses a unit axis n and angle θ, or the minimal vector omega=θn. Rodrigues' formula builds a matrix; for a small radians vector, Rv≈v+omega×v. Unit quaternions q=(v,w)=(n sin(θ/2),cos(θ/2)) have four entries with a unit-length constraint. q and −q represent the same rotation. They are convenient for smooth tracking and spherical interpolation; compose them in the same order as their corresponding matrices.

```
R = I + sin(theta)[n]_cross + (1-cos(theta))[n]_cross²
Quaternion order used here: (qx,qy,qz,qw).
Rightmost matrix acts first.
```

Worked example

Rotate (2,1) +90° counterclockwise, then translate (3,−2).

1. R(2,1)=(−1,2).
2. Add translation:(2,0).

Result: (2,0) with the stated x-right/y-up axes.

Worked example

Quaternion for +90° about z?

1. Half-angle=45°.
2. Vector part=(0,0,sin45°); scalar part=cos45°.

Result: (0,0,√2/2,√2/2), equivalent to its negative.

Exam traps

- An affine shear preserves parallelism but can change angles.
- Do not omit perspective division after a homography.
- 3D rotation order is generally noncommutative.
- Quaternion normalization and component ordering must be explicit.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §§2.1.1-2.1.3, printed pp. 40-51 (PDF pp. 66-77)

Assigned reading

Projection, calibration, and lens distortion

Separate geometry from calibration and identify where a homography is sufficient and where depth or nonlinear lens correction is needed.

Orthography drops depth: (X,Y,Z) becomes (X,Y). Scaled orthography adds a common scale, useful when an object's depth variation is small relative to its distance. Para-perspective is an affine

approximation that improves the off-axis model while retaining linear projection. Perspective divides by depth and models foreshortening.

```
Virtual plane in front of the pinhole, positive Z:
u = fx X/Z + skew Y/Z + cx
v = fy Y/Z + cy
K = [[fx,skew,cx],[0,fy,cy],[0,0,1]]
P = K [R | t], with camera coordinates = R world_point + t.
Horizontal field of view: FOV = 2 atan(W/(2f)).
```

Use matching units for image width W and f: pixels for a pixel calibration matrix, millimetres for a physical sensor. Doubling image resolution with the same field of view doubles focal length in pixels. A single projected point supplies a ray, not a unique distance. Inverse depth is often a convenient parameter; a full 4×4 representation can retain depth/disparity, and a chosen reference plane can provide a plane-plus-parallax coordinate.

A plane induces a homography between views. Pure camera rotation about one projection center also induces a homography without scene depth. General camera translation across a nonplanar scene introduces depth-dependent parallax and cannot be represented by one global homography. Object-centered projection highlights another ambiguity: for a distant shallow object, changing focal length and camera distance can yield similar image scale.

Lens distortion is separate from ideal perspective. In a common normalized-coordinate model, $r^2=x^2+y^2$, $(\hat{x},\hat{y})=(x,y)(1+k1r^2+k2r^4)$, followed by focal scaling and principal-point offset. The mapping direction and coordinates matter because other implementations parameterize the inverse. Wide-angle distortion can bend straight lines; fisheye models such as $r=f\theta$ use a different angular projection.

Worked example

Project X=2,Z=8 with f=800px,cx=320px.

1. X/Z=0.25.
2. u=800×0.25+320.

Result: u=520px.

Worked example

A 1000px image has a 90° horizontal field of view.

1. $f=1000/(2\tan 45°)$.

Result: $f=500$px.

Worked example

Distort normalized $x=0.5, y=0$ with $k_1=0.2, k_2=0$.

1. $r^2=0.25$.
2. $\hat{x}=0.5(1+0.2\times 0.25)$.

Result: $\hat{x}=0.525$.

Exam traps

- The image plane may be drawn behind or in front of the pinhole; signs must match the convention.
- Intrinsics do not replace extrinsics.
- A homography does not generally align translated views with multiple depths.
- Lens-distortion coefficients depend on coordinate normalization and mapping direction.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §§2.1.4-2.1.5, printed pp. 51-66 (PDF pp. 77-92)

Assigned reading

Lighting, reflectance, and why intensity is ambiguous

Use illumination and material models to explain brightness changes rather than equating pixel values with surface color.

Point illumination decreases with squared source distance; area sources require integrating a distribution of positions/directions and can produce soft shadows. An environment map represents

illumination by incoming direction, commonly assuming distant sources. These models determine what light arrives at a surface, not yet what the camera sees.

A BRDF describes the reflected radiance associated with incoming and outgoing directions at a surface and wavelength. Its directional variables are four angles in the general local model. Isotropic materials have no preferred in-plane direction; anisotropic brushed materials do. Reflection also depends on the cosine foreshortening factor of incoming illumination and visibility. Reciprocity interchanges the incident and reflected roles under the stated physical assumptions.

```
Normalized Lambertian example:
I = rho L max(0,n dot l), with n,l unit vectors.
Diffuse term is view-independent in the ideal model.
Phong: ambient + diffuse + specular.
Specular lobe approximately ks[max(0,view dot mirror_direction)]^p.
```

A Lambertian patch changes brightness with light direction and surface normal even if its material is unchanged. Ideal outgoing radiance is independent of viewing direction. A specular term peaks when the viewer approaches the mirror-reflection direction; increasing the Phong exponent narrows the highlight. The simple ambient term approximates illumination that the direct-source model leaves out.

The dichromatic model separates interface and body-reflection spectra with geometry-dependent magnitudes. Global illumination adds occlusion, indirect bounces, and color exchange among surfaces: ray/path tracing follows light paths, while radiosity exchanges energy among surface patches. These phenomena explain why the same local material can look different in different scenes.

Worked example

Compute I for rho=0.6,L=2,angle=60°.

1. max(0,cos60°)=0.5.
2. I=0.6×2×0.5.

Result: I=0.6 under the explicitly normalized model.

Worked example

Move a point source from distance r to 2r with angle fixed.

1. Compare$1/(2r)^2$ with$1/r^2$.

Result: Illumination becomes one quarter.

Exam traps

- Specify whether albedo/radiance conventions include a factor of π.
- Clamp the direct diffuse contribution to zero when the light is behind the surface.
- Diffuse and specular responses have different view dependence.
- A dark pixel does not uniquely identify dark material.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §§2.2.1-2.2.2, printed pp. 66-74 (PDF pp. 92-100)

Assigned reading

Optics: focus, exposure, and peripheral effects

Solve the thin-lens relation and connect aperture diameter to exposure, geometric defocus, and vignetting.

```
Thin lens: 1/zo + 1/zi = 1/f.
f-number N=f/d, so aperture diameter d=f/N.
Approximate irradiance for fixed scene: proportional to 1/N².
Natural off-axis falloff: proportional to cos(alpha)^4.
```

For a real object beyond the focal distance (zo>f), a finite object distance focuses behind the lens at zi greater than f; an object at infinity focuses at zi=f. An object inside the focal distance forms a virtual image instead. An incorrectly placed sensor plane creates a circle of confusion rather than a sharp point. Depth of field is the scene-depth range whose blur meets a chosen acceptable limit; it depends on aperture, focal length, focus distance, and that criterion.

A smaller aperture diameter (larger f-number) usually decreases geometric defocus and increases depth of field, while collecting less light. One full stop changes aperture area by a factor of two:

neighboring f-numbers are separated by roughly $\sqrt{2}$. A move from f/4 to f/8 is two stops, leaving one quarter of the irradiance with shutter time fixed. Longer exposure may compensate light loss but changes motion blur.

Real compound lenses can suffer geometric aberrations. Chromatic aberration arises because refraction depends on wavelength: transverse effects change color magnification, and longitudinal effects change color focus. Vignetting reduces peripheral brightness; natural vignetting has a $\cos^4$ off-axis relation in the simplified model, while mechanical vignetting comes from occlusion by lens components.

Worked example

f=50 mm and zo=1000 mm. Find zi.

1. $1/zi = 1/50 - 1/1000 = 19/1000$.
2. Invert the reciprocal.

Result: $zi \approx 52.631579$ mm.

Worked example

A 50 mm lens at f/4 versus f/8.

1. Diameter at f/4=12.5 mm; at f/8=6.25 mm.
2. Area ratio$=(6.25/12.5)^2=1/4$.

Result: One quarter of irradiance at f/8, with other settings fixed.

Exam traps

- Use the same length units throughout the lens equation.
- A higher f-number means a smaller aperture.
- Exposure and focus are distinct effects.
- Depth-of-field comparisons need fixed focus distance and a blur criterion.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §2.2.3, printed pp. 74-79 (PDF pp. 100-105)

Assigned reading

Digital sensing, noise, sampling, and aliasing

Track the sensing pipeline and explain why resolution, noise, and aliasing are different constraints.

Light is integrated during exposure, converted to an electronic signal, amplified, and digitized. A camera then typically demosaics, denoises/sharpens, white-balances, maps tone/gamma, and compresses. CCDs transfer charge toward an output; CMOS sensors provide local pixel readout circuitry. The important reasoning is that the display-ready pixel value can differ substantially from a linear measurement of incoming light.

Exposure time changes collected photons and motion blur. Pixel pitch determines spatial sampling density; sensing area and fill factor affect light collection. Gain/ISO rescales the signal and its noise; it does not by itself gather more photons. Noise may include fixed-pattern, dark-current, photon shot, amplifier, and quantization components. Photon-limited counts can have a Poisson model whose variance depends on intensity, while a constant-variance Gaussian approximation suits other regimes. Nominal ADC bit depth is not the same as the number of reliably useful bits.

```
Sampling frequency fs -> Nyquist limit fs/2.
To preserve a band-limited signal, sampling must exceed twice its highest frequency
under the relevant assumptions; avoid phase-degenerate equality edge cases.
Decimation by r -> new Nyquist limit = old fs/(2r).
PSF: spatial point response; OTF: Fourier transform(PSF); MTF=|OTF|.
```

Aliasing occurs when different continuous signals generate indistinguishable samples, often showing false stripes or moiré. Averaging over a sensor cell attenuates high frequencies but does not necessarily remove them all, even at a high fill factor. The overall PSF combines lens blur, sensor integration, and any optical antialiasing filter; the MTF shows how frequencies are transmitted. Downsampling after acquisition can create a new aliasing problem unless the image is filtered for the new grid first.

Worked example

fs=100 samples/unit. What is the frequency limit?

1. Nyquist limit=fs/2.

Result: 50 cycles/unit.

Worked example

Decimate an input sampled at one sample/unit by 4.

1. New fs=1/4.
2. New limit=(1/4)/2.

Result: 0.125 cycles per original input unit.

Exam traps

- Photon shot noise is not the same phenomenon as random salt-and-pepper impulse corruption.
- More ADC bits do not guarantee less sensor noise.
- Increasing gain is not equivalent to increasing photon collection.
- Filtering after aliasing cannot generally reveal which original frequency produced the samples.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §2.3 and §2.3.1, printed pp. 79-87 (PDF pp. 105-113)

Assigned reading

Color, gamma, and compression without mixing conventions

Treat color as channel responses to spectra, decode nonlinear encodings for physical arithmetic, and understand where compression loses information.

A camera channel integrates the spectrum weighted by its spectral sensitivity; it does not sample exactly one red, green, or blue wavelength. Three responses cannot uniquely specify a full spectrum. Different spectra with the same response are metamers under that condition; they may separate under another illuminant or sensor. RGB additive mixtures differ from subtractive pigment mixing.

XYZ provides a colorimetric tristimulus representation; Y relates to luminance. Dividing X,Y,Z by their sum gives chromaticity x,y,z, which sum to one and discard overall intensity. Lab applies a nonlinear reference-white-dependent mapping aimed at greater perceptual uniformity. HSV separates a hue/saturation/value description useful for manipulation but is not a physically linear light space. Channel ratios also separate relative color from common intensity scaling.

A Bayer CFA senses two green samples, one red, and one blue per typical 2×2 block. Demosaicing estimates missing channel values at every pixel. White balance can be modeled as a diagonal channel-gain transform, with more general color calibration using a 3×3 matrix.

```
Stated gamma convention: encoded e = I^(1/gamma).
Decode linear light: I = e^gamma.
Arithmetic for radiance/exposure blending should use linear light.
Chromaticity x=X/(X+Y+Z), y=Y/(X+Y+Z).
PSNR = 10 log10(peak²/MSE), with matching image range and peak.
```

Conventional JPEG commonly separates luma from chroma, may subsample chroma, transforms blocks with the DCT, quantizes coefficients, then entropy-codes them. Quantization loses precision; independent block coding can create block boundaries and ringing. The constant DCT basis gives DC, related to mean block brightness; other bases give AC detail. Lower numeric error (higher PSNR) need not imply the most pleasing perceptual result.

Source convention caution: the supplied draft's gamma wording around Eq. 3.7 conflicts with its clear Chapter 2 encoding model. All examples here state the function and invert it algebraically. The displayed Lab lightness equation also appears to omit its offset; these notes teach the role of Lab without reproducing that defective formula.

Worked example

XYZ=(2,3,5): find chromaticity.

1. Sum=10.
2. Divide each coordinate by 10.

Result: (x,y,z)=(0.2,0.3,0.5).

Worked example

Encode as e=√I, observe e=0.5.

1. Invert: $I=e^2$.

Result: I=0.25.

Worked example

Peak=1, MSE=0.01.

1. Compute 10 log10(1/0.01).

Result: PSNR=20 dB.

Exam traps

- Encoded RGB is not automatically linear radiance.
- Equal channel offsets generally change color ratios.
- Chromaticity is undefined for zero total signal.
- Use peak 1 for normalized images and peak 255 for 8-bit images.
- PSNR measures squared-error fidelity, not every perceptual artifact.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 2, §§2.3.2-2.3.3, printed pp. 87-101 (PDF pp. 113-127)

Assigned reading

Point operations, alpha compositing, and tone distributions

Separate linear gain from affine offset/clipping, use alpha correctly, and compute CDF-based tone mappings with an explicit convention.

A point operation changes a pixel using the corresponding input pixel, possibly with globally estimated parameters. g=af+b uses gain and bias. Multiplication by fixed a is linear; nonzero b makes the mapping affine; clipping makes the full bounded operation nonlinear. A shared RGB scale preserves normalized channel ratios before clipping, whereas equal offsets or separate gains can change perceived color.

```
Straight foreground F over opaque B: C=alpha F+(1-alpha)B.
Premultiplied foreground P=alpha F: C=P+(1-alpha)B.
Histogram h(i)=number of pixels at level i.
CDF(i)=sum_(j<=i) h(j)/N.
Example equalization convention: s(i)=(L-1)CDF(i), before rounding.
```

Matting estimates foreground color, background color, and alpha from observations; compositing uses those quantities to make a new image. Alpha is coverage/opacity: one is fully foreground and zero is fully background. Fractional alpha supports fine boundaries. Premultiplied storage makes filtering and resampling RGBA convenient, but its color must not be multiplied by alpha twice. Physical light mixtures should be computed in linear light.

A histogram discards spatial arrangement but describes intensity occupancy. Equalization remaps by a CDF to spread values; discrete finite images need not become exactly flat. Implementation conventions differ in whether they subtract the first nonzero CDF and how they round. Global equalization can overemphasize noise or produce an unwanted tone distribution, so partial mappings blend with the identity. Independent local blocks can make seams; adaptive histogram equalization interpolates local transfer functions, and contrast limits reduce excessive amplification.

Worked example

f=180,g=clip(1.5f+20,0,255).

1. Unclipped value=290.
2. Clamp to8-bit maximum.

Result: 255.

Worked example

Straight F=0.8,B=0.2,alpha=0.25.

1. Foreground contribution=0.20.
2. Background contribution=0.15.

Result: C=0.35.

Worked example

Histogram[1,2,1,0], L=4; find s(1) under s=3CDF.

1. N=4; CDF(1)=3/4.
2. s(1)=3×3/4.

Result: 2.25 before rounding; no CDFmin subtraction is used.

Exam traps

- A histogram contains no object position information.
- Premultiplied and straight color values are different representations.
- Do not assume every library uses the same equalization convention.
- An operation can be pointwise while its parameters were estimated globally.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.1, printed pp. 109-119 (PDF pp. 135-145)

Assigned reading

Linear, separable, directional, and recursive filtering

Read a kernel with explicit offsets, reason about frequency and border effects, and recognize efficient implementations.

```
Correlation: g(i,j)=sum_(k,l) f(i+k,j+l) h(k,l).
Convolution: g(i,j)=sum_(k,l) f(i-k,j-l) h(k,l).
Separable kernel: K=v h^T; K×K direct work≈K², two passes≈2K.
Unsharp mask: sharpened=f+lambda(f-blurred).
Directional derivative: G_u=cos(theta)G_x+sin(theta)G_y.
```

Convolution reverses the kernel relative to correlation; symmetric kernels produce the same result, but an antisymmetric derivative changes sign. Fixed neighborhood filtering is linear, and it is shift invariant on an unbounded grid or where boundary rules do not interfere. Kernels summing to one preserve a constant signal under suitable boundaries; derivative/band-pass kernels have zero response to a constant.

Zero padding can darken borders, replicate padding repeats edge values, wrapping assumes periodicity, and reflection depends on the exact endpoint convention. State the border rule before calculating. A separable rank-one kernel is an outer product: Gaussian approximations, box kernels, and Sobel constructions often exploit this. A nonseparable kernel may be approximated by a sum of separable SVD terms.

Smoothing suppresses high frequencies and can reduce noise, but also softens edges. Unsharp masking adds a residual and may amplify noise or produce halos. Sobel x combines differentiation across x with smoothing across y and therefore responds to vertical edges. Gaussian derivative

families provide smoother band-pass analysis; LoG combines smoothing with the Laplacian, and steerable bases make many directions cheap to evaluate.

An integral image stores running sums so that a rectangle sum needs four lookups with inclusion-exclusion. Accumulator range must be large enough for many pixels. FIR filters use a finite input footprint. Recursive/IIR filters use previous outputs and can have an indefinitely extended impulse response. These are implementation/response distinctions, not a guarantee that a filter is linear or nonlinear in every design.

Worked example

Neighborhood[2,4,8], kernel[−1,0,1].

1. Correlation=(-1)2+0(4)+1(8)=6.
2. Convolution=(-1)8+0(4)+1(2)=−6.

Result: Correlation6; convolution−6 under centered offsets.

Worked example

Rectangle integral lookups40,12,9,3.

1. 40−12−9+3.

Result: 22.

Worked example

7×7 separable filtering cost.

1. Two7-tap passes use14 terms instead of49.

Result: 14 terms per output pixel, ignoring overhead.

Exam traps

- Kernel symmetry can hide a correlation/convolution mistake.
- Derivative direction and edge orientation are perpendicular.
- Borders can violate a casual claim of perfect shift invariance.
- Clipping after a linear filter changes the full operator.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.2, printed pp. 119-131 (PDF pp. 145-157)

Assigned reading

Nonlinear denoising, morphology, and regions

Choose an operator by its assumptions about outliers, edges, binary shape, and adjacency.

A median chooses the middle neighborhood value and is useful against isolated impulse outliers. A mean averages all samples and can be better for independent small zero-mean noise when edges are absent. A trimmed mean removes extreme values before averaging. Weighted absolute-deviation minimization gives a weighted median; squared-deviation minimization gives a weighted mean. These assumptions explain the different behavior, rather than one filter being universally best.

```
Bilateral weight(i,j)=spatial_weight(i,j)×range_weight(i,j).
spatial_weight=exp(−distance²/(2 sigma_d²)).
range_weight=exp(−intensity_difference²/(2 sigma_r²)).
Output=sum(weights×input_values)/sum(weights).
```

Large intensity differences reduce bilateral mixing across edges; the range weights depend on the image, making the usual filter nonlinear. A very large sigma_r approaches ordinary spatial Gaussian smoothing. Joint/cross-bilateral filtering uses another image for the range weights while averaging the input values. Guided filtering models a local affine relationship to a guide. Image alignment and guide relevance matter. Repeating edge-dependent local averaging relates to anisotropic diffusion, but excessive iteration can destroy detail; the process needs an appropriate stopping rule or data fidelity.

Binary operation (foreground=1)	Meaning
Dilation	Output1 if any foreground appears under the structuring element; grows foreground.
Erosion	Output1 if all required samples are foreground; shrinks foreground.
Opening	Erosion then dilation; removes structures too small to survive the element.
Closing	Dilation then erosion; fills sufficiently small gaps/holes.
Majority	Threshold the foreground count near half the element size; define an even-size tie policy if necessary.

A distance transform gives the distance to the nearest specified background/target set. State whether zeros or ones form the target and which metric is used. Manhattan distance sums absolute coordinate differences; Euclidean uses their squared sum and root. Signed distances add an inside/outside sign convention. Connected components depend on N4 versus N8 adjacency; compute region area, centroid, and moments after labeling. A diagonal contact can join components under N8 and leave them separate under N4.

Worked example

Median of[10,11,250,12,9].

1. Sort[9,10,11,12,250].

Result: 11.

Worked example

Values10,20 with bilateral weights1,0.5.

1. Weighted sum=20; normalization=1.5.

Result: 13.333333.

Worked example

Nearest zero is at(0,2), query(3,4), Manhattan metric.

1. 3+2.

Result: 5.

Worked example

Pixels(0,0),(1,1) only.

1. No horizontal/vertical path exists.
2. A diagonal connection exists.

Result: Two N4 components; one N8 component.

Exam traps

- Photon-count shot noise and impulse corruption are different models despite loose terminology in the draft.
- Normalized bilateral weights must sum to one after division.
- Foreground convention and the structuring-element shape change morphology results.
- Distance to background is not distance to the origin unless the origin is the nearest target.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.3, printed pp. 131-142 (PDF pp. 157-168)

Assigned reading

Fourier reasoning for filters and restoration

Use frequency response to explain blur, sharpening, transform computation, and what lost information cannot be restored.

An LSI filter maps a sinusoid into a sinusoid of the same frequency with a changed amplitude and phase. The Fourier transform tabulates this response. Low frequencies describe slow variations; high

frequencies describe rapid spatial changes. The complex phase matters for where structures occur, so magnitude alone is generally insufficient to reconstruct an image.

```
Forward convention used in the textbook:
F(k)=(1/N) sum_n f(n) exp(−j2pi kn/N).
Then DC F(0)=mean(f).
Many libraries instead use an unnormalized forward transform and a 1/N inverse.
DFT direct: O(N²); FFT: O(N log N), the same mathematical transform.
```

Convolution corresponds to multiplication in frequency, with the scale factor depending on normalization. Under an unnormalized forward DFT and normalized inverse, circular convolution gives G=FH. A finite FFT assumes periodic extension; pad to at least N+M−1 for the full linear convolution of 1D arrays of lengths N and M. For 2D images the idea applies along both axes.

The DCT uses real cosine bases and is useful for blocks because natural patches often concentrate energy in low-frequency coefficients. Its DC component relates to average brightness. Filtering can be studied in either spatial or frequency terms: smoothing attenuates rapid variation; derivative filters suppress constant values and emphasize certain variation; a band-pass filter suppresses both very low and very high frequencies.

Restoration must consider signal and noise together. Dividing by a tiny blur response amplifies noise; no stable division exists where the response is zero. The reading introduces the statistical idea of Wiener restoration but removes its derivation in this edition; it is background here, not an added derivation requirement. PSNR compares output with a known clean reference using MSE; perceptual metrics assess other aspects of similarity. Apparent sharpening does not prove the original physical detail was recovered.

Worked example

Normalized DFT DC for[2,4,6,8].

1. At k=0 the exponential is1.
2. Divide sum20 byN=4.

Result: 5.

Worked example

What FFT padding is sufficient for lengths4 and3?

1. Full linear support length=4+3−1.

Result: At least6 samples.

Worked example

Unsharp pixel100, blur80, gain0.5.

1. Residual20.
2. 100+0.5×20.

Result: 110 before clipping.

Exam traps

- State transform normalization before a numeric DFT calculation.
- An FFT is an algorithm, not a different representation from the DFT.
- DFT multiplication is circular until padding is handled.
- Amplitude boost can amplify noise and cannot uniquely recover erased information.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.4, printed pp. 142-149 (PDF pp. 168-175)

Assigned reading

Interpolation, pyramids, wavelets, and multiband blending

Keep sampling and reconstruction operators explicit and use scale-dependent representations for analysis and blending.

Interpolation estimates values between available samples; it increases output grid density without creating new measured information. Linear interpolation is simple but can soften or create derivative creases. Cubic and windowed-sinc choices trade sharpness, smoothness, cost, overshoot, and ringing.

An interpolating kernel passes through source samples; an approximating B-spline reconstruction need not. Decimation first filters for the smaller Nyquist range and then reduces the sample rate.

```
Gaussian octave pyramid: G_(l+1)=downsample2(lowpass(G_l)).
Laplacian residual: L_l=G_l−expand(G_(l+1)).
Reconstruction: G_l=L_l+expand(G_(l+1)).
Keep the coarsest G and every residual L for exact reconstruction.
Each octave has1/4 as many pixels as its parent.
```

A Gaussian pyramid supports multi-scale detection and coarse-to-fine search. A Laplacian pyramid adds residual bands so the exact fine image can be recovered before quantization when the same expansion operator is used. Its name does not make each residual exactly a mathematical LoG: a difference of smoothed scales produces a DoG-like band-pass effect. Half-octave/quincunx representations provide intermediate scales with different grids.

Wavelets localize structure in space, scale, and often orientation. A critically sampled separable 2D stage produces LL,LH,HL,HH, each with one quarter of the pixels. Recurse on LL and retain detail bands; the total coefficient count can equal the source count, whereas regular pyramids are overcomplete. Some wavelet/steerable families deliberately use redundancy for better shift/orientation behavior. Specify axis naming before attaching horizontal/vertical edge names to LH and HL.

Lifting splits even and odd samples, predicts one sequence from the other and stores the residual, then updates the coarse sequence. Reverse the operations and signs to reconstruct; the predictor need not be perfect. Multiband blending builds Laplacian pyramids of the images and a Gaussian pyramid of the mask. Coarse color variations blend broadly, while fine texture blends locally; reconstruct the weighted bands afterward.

Worked example

Linear interpolation of 10 and 30 at t=0.25.

1. 0.75×10+0.25×30.

Result: 15.

Worked example

256×256 input, octave level 2.

1. Level 1=128×128.
2. Level 2=64×64.

Result: 4096 pixels.

Worked example

G=50 and expanded coarse estimate 44.

1. Residual=50−44.
2. Reconstruct with 6+44.

Result: Residual 6; exact original 50.

Worked example

Infinite octave pyramid sample count.

1. N(1+1/4+1/16+…).
2. Geometric sum=N/(1−1/4).

Result: 4N/3, ignoring integer rounding and finite termination.

Exam traps

- Upsampling does not add new measured scene information.
- An antialiasing filter must match the decimation factor.
- DoG and LoG are related but not identical at arbitrary scales.
- Drop the coarsest Gaussian level and exact Laplacian reconstruction is lost.
- Some wavelet families are redundant, so do not universalize the critical-sampling count.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.5, printed pp. 149-167 (PDF pp. 175-193)

Assigned reading

Warping, antialiasing footprints, and morphing

Map destination coordinates back to source samples, handle minification, and distinguish geometric alignment from appearance blending.

A point operator changes an image's range; a geometric warp changes its domain. If the forward coordinate map is $x' = h(x)$, a destination-driven implementation samples $g(x') = f(h^{-1}(x'))$. Forward mapping pushes source samples onto target locations; rounding can create overlaps, holes, and flicker. Splatting distributes samples but needs weighting/normalization and can blur. Inverse warping queries every destination pixel where its source map is defined, then handles noninteger coordinates by interpolation and out-of-range coordinates by a stated border rule.

```
Bilinear weights at relative(x,y)in[0,1]²:
w00=(1-x)(1-y), w10=x(1-y), w01=(1-x)y, w11=xy.
MIP octave level for source-footprint scale r: l=log2(r).
Fractional l can blend adjacent levels after within-level sampling.
```

Inverse warping solves sampling coverage, not every aliasing problem. One destination pixel can cover a large or elongated source footprint under minification. The local Jacobian gives an affine approximation to the transform and the directional sampling rates. MIP-maps use prefiltered resolutions for efficient scalar scale selection. Trilinear MIP sampling blends two bilinearly sampled levels. Elliptical/anisotropic filtering can better follow a footprint that compresses one axis more than another. Multi-pass 1D resampling/shearing can improve efficiency but still needs careful antialiasing.

Global transformations have a few parameters; a face-expression edit often needs local deformation. A triangular mesh interpolates displacement by a per-triangle affine map from three non-collinear correspondences. Quadrilateral grids, smooth spline fields, sparse control points, or line correspondences provide other warp models. Folded or degenerate meshes can make the mapping invalid, so correspondence geometry matters.

Cross-dissolving only mixes corresponding pixel intensities and can duplicate unaligned features. A feature-based morph first defines intermediate landmarks, warps both images into that common geometry, and then blends appearance. At time t the usual position interpolation and appearance weights are $1-t$ and t. Geometry and color interpolation are separate calculations.

Worked example

Forward $x' = 2x + 3$, destination 11.

1. Solve $x = (11 - 3)/2$.

Result: Source $x = 4$.

Worked example

Corners 0,10,20,30 at (0,0),(1,0),(0,1),(1,1); sample (0.25,0.5).

1. Weights=0.375,0.125,0.375,0.125.
2. Weighted sum=1.25+7.5+3.75.

Result: 12.5.

Worked example

A destination footprint spans 8 source widths.

1. $l = \log_2(8)$.

Result: MIP level 3.

Worked example

Morph at t=0.5.

1. Interpolate common midpoint landmarks.
2. Warp both inputs to that geometry.
3. Mix aligned intensities equally.

Result: Aligned midpoint shape with a 50/50 appearance blend.

Exam traps

- Use the inverse coordinate transform when pulling destination samples.
- Interpolation and antialiasing are related but serve different needs under minification.
- Bilinear corner order and axes must be stated.
- One global homography cannot represent every local deformation.
- Blending alone does not align features.

▶ **Check your understanding**

Sources: Szeliski, 2nd-edition draft, Ch. 3, §3.6, printed pp. 168-178 (PDF pp. 194-204)

Source corrections and calculation conventions

Lecture 3 slide 84

Uniform-score cross-entropy formula has an incorrect minus sign. $L=-\ln(1/C)=\ln(C)$, consistent with its positive log(10) numerical example.

Lecture 3 slides 17–18

Score-scaling identity assumes bias is ignored. For $f(x)=Wx+b$, $f(cx)=cf(x)+(1-c)b$; the simple identity is valid with $b=0$.

Lecture 3 slides 81–82

Minimum-zero wording needs the finite-logit qualification. With more than one class and finite logits, cross-entropy is positive and has infimum zero as p_y approaches one.

Lecture 5 slide 11

Floor formula is inconsistent with the exact-fit table entry for $W=7,K=3,S=3,P=0$. Standard floor sizing gives two fully fitting windows. An exact-tiling convention can reject the remainder, but that is a different convention.

Lecture 5 slides 4,14

Flattening wording suggests literal information loss. Known flattening preserves values and ordering; an ordinary FC layer lacks enforced locality and sharing.

Lecture 5 slides 5,18

Translation equivariance and small-shift pooling tolerance can be mistaken for perfect classifier invariance. Equivariance shifts feature maps; invariance fixes outputs. Padding, stride, boundaries and the classifier limit exact guarantees.

Trask Ch. 4, printed pp. 50,56–59,70–71

The displayed squared error and the named derivative use different factor conventions. For E=(p−y)^2 the gradient is 2(p−y)x. The book uses (p−y)x, matching half-squared loss or a rescaled learning rate.

Trask Ch. 5, printed pp. 91–93

Several printed multi-output snippets use weights rather than residuals, overwrite delta, or orient the outer product inconsistently. For column-vector p=Wx with output rows, maintain delta[i]=p[i]−y[i] and use outer(delta,input), then update W simultaneously.

Lecture 4 slide 32

Fixed finite-difference step and relative-error thresholds are presented as universal rules. They are heuristics. Account for dtype, magnitude, near-zero derivatives, cancellation and nonsmooth branch changes.

Lecture 5 slide 24

Monitoring test accuracy can be misread as authorization to tune repeatedly on the final test set. Use validation performance for selection; reserve test assessment after choices are fixed.

Lecture 5 slide 38

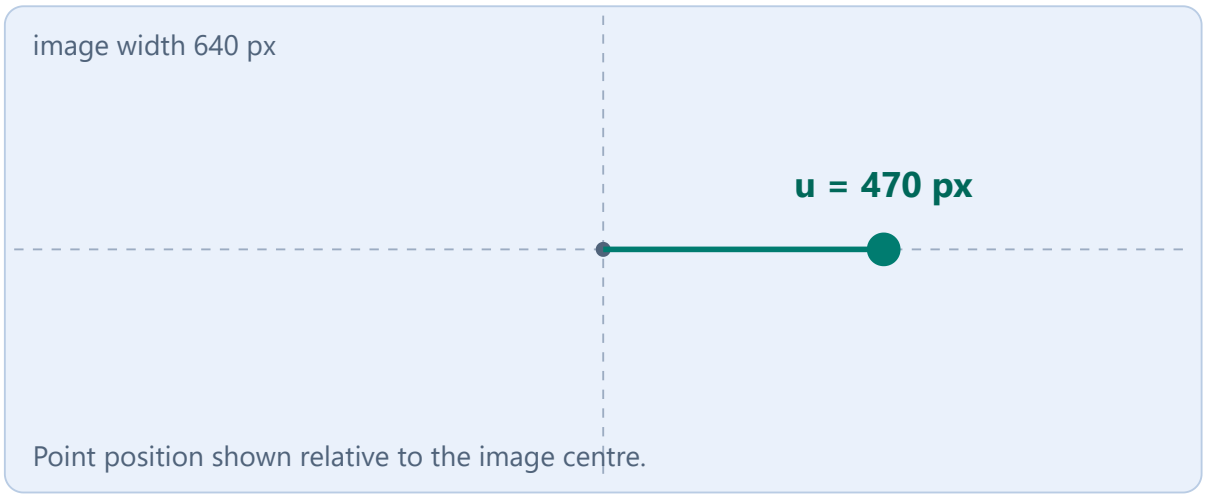
The small course experiment can be misread as a universal architecture ranking. Treat its accuracy figures as specific to the reported subset, CPU budget and short training procedure.

Question prompts specify conventions for loss normalization, indexing, gamma exponents, convolution versus correlation, image shapes and rounding.

VISUAL CALCULATION

Perspective divides by depth.

Hold an object's camera-space X coordinate fixed. Increasing Z moves its image toward the principal point.



principal point (320,180)

$$
\begin{aligned}
u &= f_x \, X/Z + c_x \\
&= 600 \times 1 / 4 + 320 \\
&= \mathbf{470 \text{ pixels}} \\
v &= 180 \text{ pixels}
\end{aligned}
$$

Source: Szeliski Chapter 2, §2.1.4. This diagram shows image coordinates, not a physical sensor behind the pinhole.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

One output is a weighted patch.

Select an output cell to trace its 3×3 patch. Row and column indices start at zero. Valid mode, stride 1, one channel, bias 0.

Input 5×5

1	2	0	1	3
0	1	3	2	1
2	1	0	0	1
1	0	2	1	0
3	1	1	2	2

Kernel 3×3

-1	0	1
-2	0	2
-1	0	1

Output 3×3

3	0	0
0	0	-2
-2	2	-2

```
output[0,0] = sum of 9 products
1×(-1) + 2×(0) + 0×(1)
+ 0×(-2) + 1×(0) + 3×(2)
+ 2×(-1) + 1×(0) + 0×(1)
= 3
```

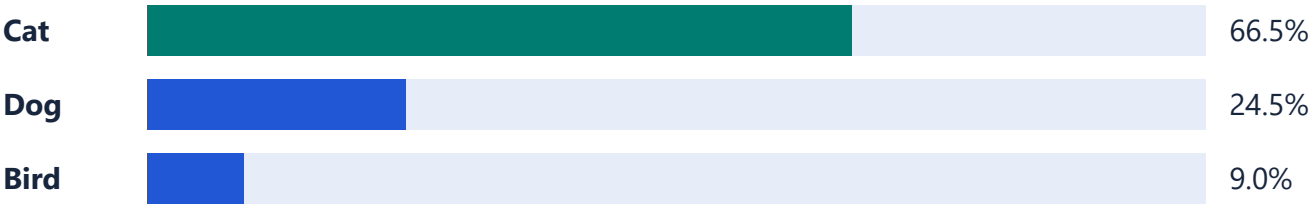
Sources: Szeliski Chapter 3, §3.2; Lecture 5, slides 6–8. Border mode and data type can change real library outputs.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

A correct class can still carry a loss.

Compare argmax, softmax cross-entropy, and multiclass hinge loss for the same scores. Natural logarithm; hinge margin Δ=1.



Teal marks the true class. If scores tie, this illustration uses the first listed class.

```
effective scores = [2, 1, 0]
prediction = Cat (correct)
softmax p(true) = 0.66524
cross-entropy = −ln p(true) = 0.40761
hinge loss = Σⱼ≠y max(0, sⱼ−sᵧ+1) = 0
```

Adding the same constant changes neither softmax nor hinge loss. Multiplying scores by a positive factor preserves argmax but usually changes confidence and loss.

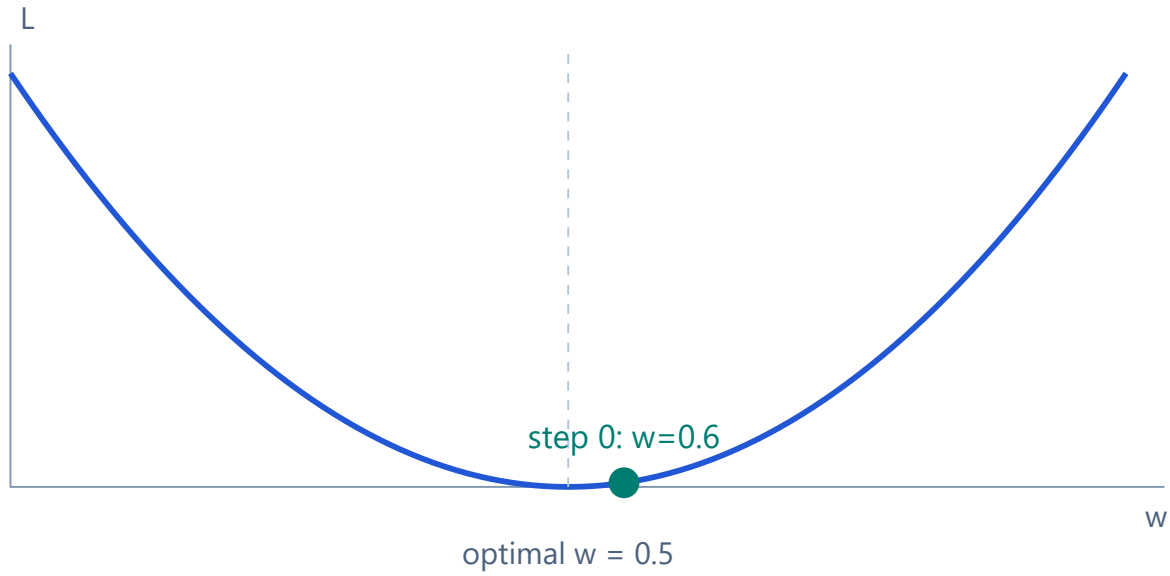
Source: Lecture 3, pages 39–57 and 70–89. Softmax gives normalized model probabilities; calibration requires empirical checking.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

Follow one gradient update.

One training pair: x=2, target y=1. No bias. Loss $L=\frac{1}{2}(wx-y)^2$. For this quadratic, a fixed rate converges when $0<\alpha<0.5$.




```
current w = 0.6
ŷ = 2w = 1.2; error = 0.2
L = ½(error)² = 0.02
gradient = error × x = 0.4
next w = w − α × gradient
       = 0.6 − 0.1 × (0.4)
       = 0.56
```

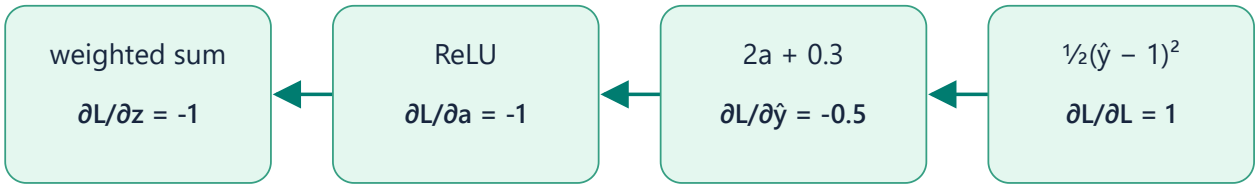
Sources: Trask Chapters 4–5; Lecture 4 slide 16. The rate threshold here is specific to x=2 and the half-squared loss.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

Multiply local derivatives, add branch contributions.

One hidden ReLU, then a scalar linear output. x=[2,−1], w=[0.5,1], output weight v=2, output bias c=0.3, target y=1. $L=\frac{1}{2}(\hat{y}-y)^2$.



Loss sensitivity flows back through the graph.

```
∂L/∂ŷ = ŷ − y = -0.5
∂L/∂a = (∂L/∂ŷ) × v = -1
ReLU′(z) = 1; ∂L/∂z = -1
∂L/∂w = [-2, 1]
∂L/∂b = -1
∂L/∂v = (∂L/∂ŷ) × a = -0.05
∂L/∂c = -0.5
```

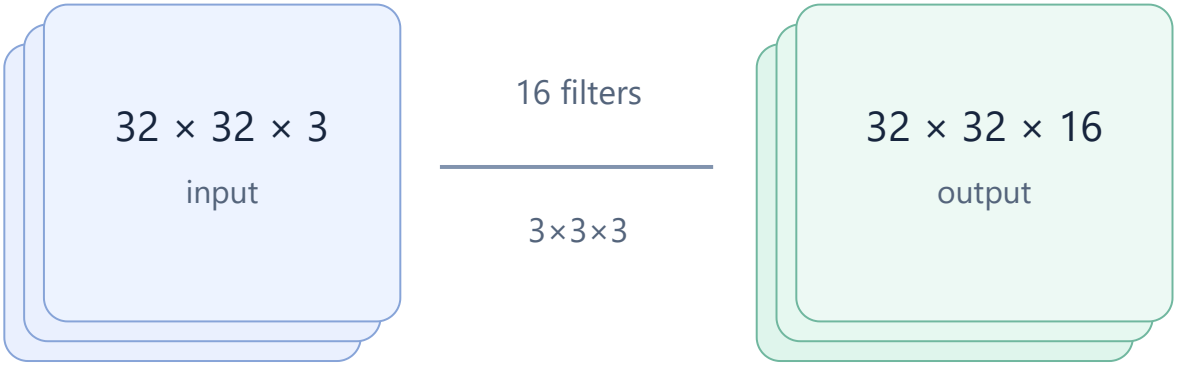
Source: Lecture 4 slides 25–30. This is an additional numerical example; the lecture's spam example uses sigmoid plus binary cross-entropy.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

Keep activation size separate from parameter count.

Standard dense square convolution, dilation 1, symmetric zero padding, floor output convention, one bias per output channel.




```
O = floor((32 + 2×1 − 3)/1) + 1
  = 32
parameters = 16 × (3 × 3² + 1)
           = 448
output activations = 16,384
weight multiplies (excluding bias) = 442,368
```

Changing image resolution changes the number of activations and multiply operations. It does not change this layer's learned parameter count.

Receptive-field example

```
Input: r=1, j=1
1. 3×3 conv, S=1: r=3, j=1
2. 2×2 pool, S=2: r=4, j=2
3. 3×3 conv, S=1: r=8, j=2
Current receptive field: 8×8 input pixels
```

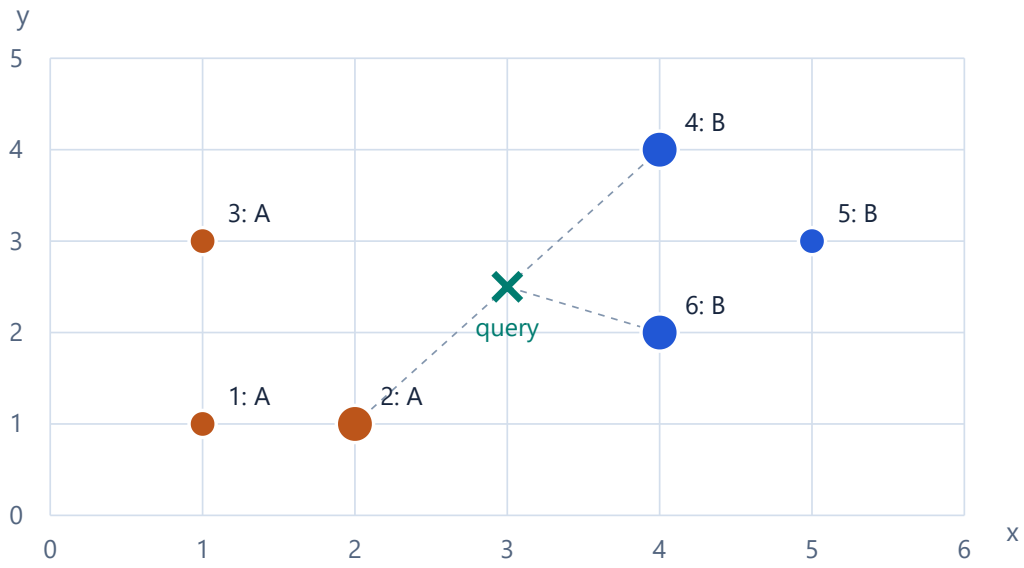
Source: Lecture 5 slides 9–14. Grouped and depthwise convolution use different parameter formulas.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

The neighbours decide the vote.

Change the query point, k and metric. This is a two-feature illustration. It shows why feature scale and the chosen distance matter.




```
L2 = √(Δx² + Δy²)
nearest 3: 6(B, d=1.11803), 2(A, d=1.80278), 4(B, d=1.80278)
votes: A=1, B=2
prediction = class B
```

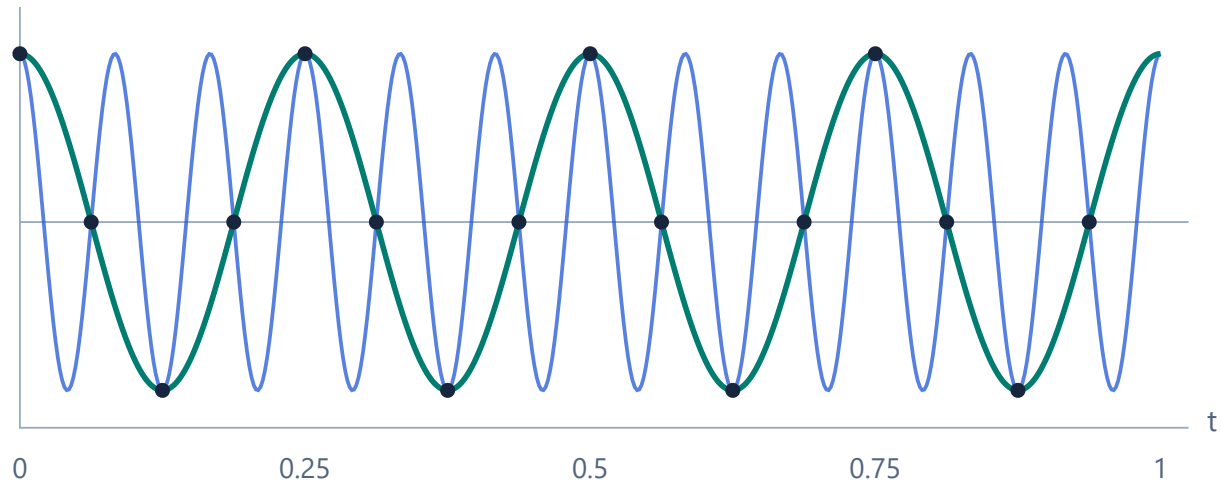
Source: Lecture 2, nearest neighbours and kNN. Choose k and any learned feature scaling on validation data, without using final test labels.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.

VISUAL CALCULATION

Different frequencies can produce identical samples.

A one-dimensional slice models a repeated image pattern. Signal $x(t)=\cos(2\pi ft)$, sampled at $f_s=16$ samples per unit t. The plotted dense curves show possible signals, not extra measured pixels.



16 measured samples on $0 \le t < 1$

```
f_s = 16; Nyquist limit = 8
original frequency = 12
indistinguishable sampled frequency = 4
Aliasing: the samples cannot distinguish 12 from 4.
```

Sources: Szeliski §2.3.1 and §3.5.2. This is a noiseless periodic cosine example. The general Nyquist limit is $f_s/2$.

This is the default worked state. Change values in the website's Visual lab to explore alternatives.